

4.5 Manipulating LINKED LISTS

- Another useful operation is one which inverts a chain.
- This routine is especially interesting because it can be done "in place" if we make use of 3 pointers.

procedure *INVERT*(*X*)

 // a chain pointed at by *X* is inverted so that if $X = (a_1, \dots, a_m)$
 then after execution $X = (a_m, \dots, a_1)$ //

$p \leftarrow X; q \leftarrow 0$

while $p \neq 0$ **do**

$r \leftarrow q; q \leftarrow p$ // *r* follows *q*; *q* follows *p* //

$p \leftarrow \text{LINK}(p)$ // *p* moves to next node //

$\text{LINK}(q) \leftarrow r$ // link *q* to previous node //

end

$X \leftarrow q$

end *INVERT*

- Try this algorithm out on at least 3 examples (the empty list, and lists of length 1 and 2)
- For a list of $m \geq 1$ nodes, the **while** loop is executed m times and so the computing time is linear or $O(m)$.

INVERTING LINKED LIST

X →

Z =

Invert(X)

p=X=4; q=0

→ While p!=0 4!=0 do

r=0; q=p q=4

p=LINK(p) p=LINK(4) p=5

LINK(q)=r LINK(4)=0

→ While p!=0 5!=0 do

r=4; q=p q=5

p=LINK(p) p=LINK(5) p=6

LINK(q)=r LINK(5)=4

→ While p!=0 6!=0 do

r=5; q=p q=6

p=LINK(p) p=LINK(6) p=0

LINK(q)=r LINK(6)=5

→ While p!=0 0!=0 X FALSE

X=q X=6

Y= 7→8→9

procedure INVERT(X)

// a chain pointed at by X is inverted so that if $X = (a_1, \dots, a_m)$
then after execution $X = (a_m, \dots, a_1)$ //

$p \leftarrow X; q \leftarrow 0$

while $p \neq 0$ do

$r \leftarrow q; q \leftarrow p$ // r follows q; q follows p //

$p \leftarrow \text{LINK}(p)$ // p moves to next node //

$\text{LINK}(q) \leftarrow r$ // link q to previous node //

end

$X \leftarrow q$

end INVERT

4

5 → 4

6 → 5 → 4

Circular

- Another useful subroutine is one which concatenates two chains X and Y .

```

procedure CONCATENATE( $X, Y, Z$ )
  //  $X = (a_1, \dots, a_m), Y = (b_1, \dots, b_n), m, n \geq 0$ , produces a new chain
   $Z = (a_1, \dots, a_m, b_1, \dots, b_n)$  //
   $Z \leftarrow X$ 
  if  $X = 0$  then [ $Z \leftarrow Y$ ; return]
  if  $Y = 0$  then return
   $p \leftarrow X$ 
  while  $LINK(p) \neq 0$  do           // find last node of  $X$  //
     $p \leftarrow LINK(p)$ 
  end
   $LINK(p) \leftarrow Y$            // link last node of  $X$  to  $Y$  //
end CONCATENATE

```

- This algorithm is also linear in the length of the first list.
- From an aesthetic point of view it is nicer to write this procedure using the case statement in SPARKS. This would look like:

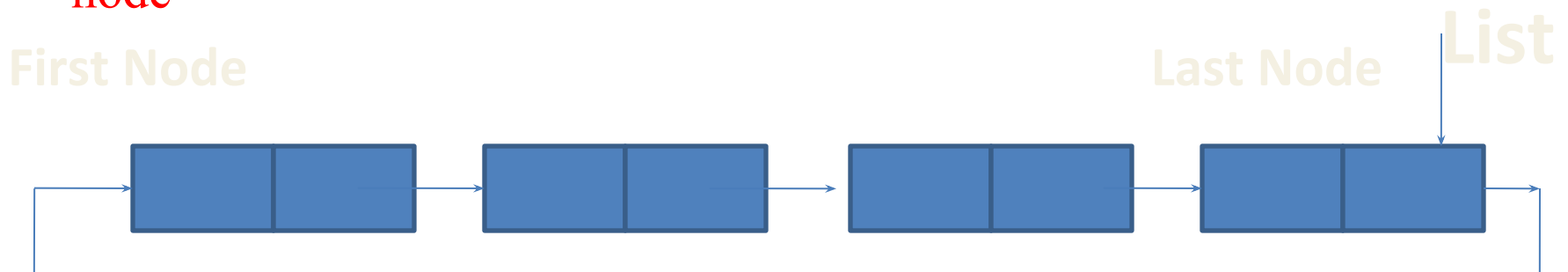
```

procedure CONCATENATE( $X, Y, Z$ )
  case
    :  $X = 0$  :  $Z \leftarrow Y$ 
    :  $Y = 0$  :  $Z \leftarrow X$ 
    : else :  $p \leftarrow X; Z \leftarrow X$ 
              while  $LINK(p) \neq 0$  do
                 $p \leftarrow LINK(p)$ 
              end
               $LINK(p) \leftarrow Y$ 
  end
end CONCATENATE

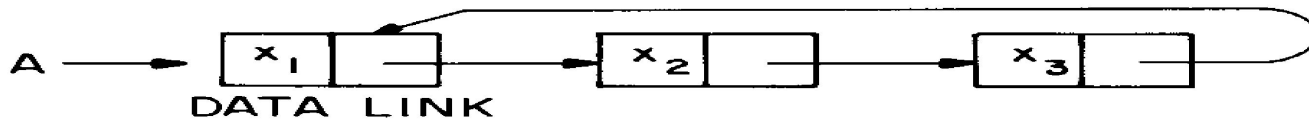
```

Circular List

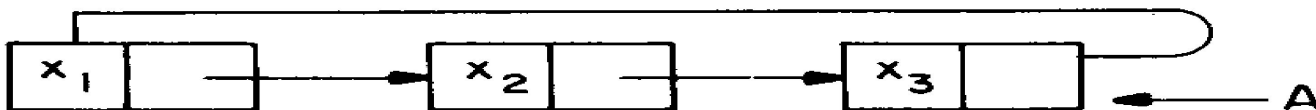
- Given a pointer p to a node in linear list, we can't reach any of the nodes that **precede node(p)**
- If a list is traversed, **external pointer** to the list must be **preserved** to reference the list again
- To overcome these problems, make the next field of last node contains a pointer back to the first node **instead of null**.
- This type of list is called as **circular linked list**.
- In CLL (Circular Linked List) there is exactly **no “first” and “last” node**



- Now let us take another look at circular lists like the one below:



- $A = (x_1, x_2, x_3)$.
- To insert a new node at the front of this list we have to change the LINK field of the node containing x_3 .
- This requires to move down the entire length of A until we find the last node.
- It is more convenient if the name of a circular list points to the last node rather than the first, for example:



- now we can write procedures which insert a node at the front or at the rear of a circular list and take a fixed amount of time.

```

procedure INSERT_FRONT(A, X)
    //insert the node pointed at by X to the front of the circular list
    A, where A points to the last node//
    if A = 0 then [A ← X
                    LINK(X) ← A ]
    else [LINK(X) ← LINK(A)
          LINK(A) ← X]
end INSERT_FRONT

```

- To insert *X* at the rear, one only needs to add the additional statement *A* → *X* to the **else** clause of *INSERT_FRONT*.
- As a last example of a simple procedure for circular lists, we write a function which determines the length of such a list.

```

procedure LENGTH(A)
    //find the length of the circular list A//
    i ← 0
    if A ≠ 0 then [ptr ← A
                    repeat
                        i ← i + 1; ptr ← LINK(ptr)
                    until ptr = A ]

    return (i)
end LENGTH

```



If $A=0$ X

Else

$Link(X)=Link(A)$ $LINK(7)=LINK(6)=4$

$LINK(6)=7$

procedure *INSERT_FRONT*(A, X)

//insert the node pointed at by X to the front of the circular list
 A , where A points to the last node //

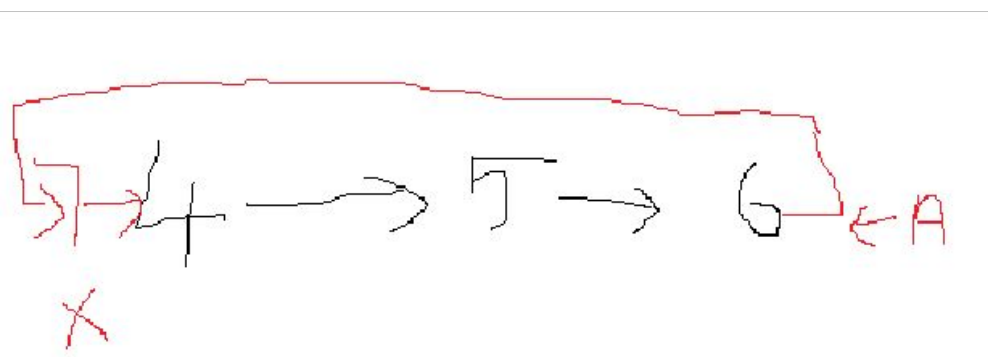
if $A = 0$ then [$A \leftarrow X$

$LINK(X) \leftarrow A$]

else [$LINK(X) \leftarrow LINK(A)$

$LINK(A) \leftarrow X$]

end *INSERT_FRONT*



Insertion at rear

If $A=0$ X

Else

$Link(X)=LINK(A)$ $LINK(7)=LINK(6)=4$

$LINK(6)=7$

$A=X$ $A=7$



LENGTH (of above CLL)

$i=0$

if $A \neq 0$ then $[ptr=7$

repeat

$\rightarrow i=0+1=1$

$ptr=LINK(ptr)$ $ptr=LINK(7)=4$

$\rightarrow i=2$

$Ptr=LINK(4)=5$

$\rightarrow i=3$

$Ptr=LINK(5)=6$

$\rightarrow i=4$

$Ptr=LINK(6)=7$

Until $ptr=A$ $PTR=7$

Return $i=4$

Length=4

procedure *INSERT_FRONT*(A, X)

//insert the node pointed at by X to the front of the circular list
 A , where A points to the last node //

if $A = 0$ then $[A \leftarrow X$

$LINK(X) \leftarrow A]$

else $[LINK(X) \leftarrow LINK(A)$

$LINK(A) \leftarrow X]$

end *INSERT_FRONT*

To insert X at the rear, one only needs to add the additional statement $A \leftarrow X$ to the **else** clause of *INSERT_FRONT*.

4.8 Doubly Linked Lists (DLL)

- So far we have been working chiefly with **singly linked linear lists**. For some problems these would be too restrictive.
- One difficulty with these lists is that if we are pointing to a **specific node**, say P , then we can easily **move only in the direction** of the links.
- The only way to find the **node which precedes P** is to **start back at the beginning** of the list.
- The same problem arises when one wishes to **delete an arbitrary node** from a singly linked list.
- In order to easily delete an arbitrary node one must know the **preceding node**.
- If we have a problem where moving in **either direction** is often necessary, then it is useful to have **doubly linked lists**.
- Each node now has two link fields, one linking in the **forward direction** and one in the **backward direction**.

- A node in a doubly linked list has at least 3 fields, say
 - DATA,
 - LLINK (left link) and
 - RLINK (right link).
- A doubly linked list may or may not be circular. A sample doubly linked circular list with 3 nodes is given in figure 4.13.
- Besides these three nodes a special node has been added called a head node.
- As was true in the earlier sections, head nodes are again convenient for the algorithms. The DATA field of the head node will not usually contain information. Now suppose that P

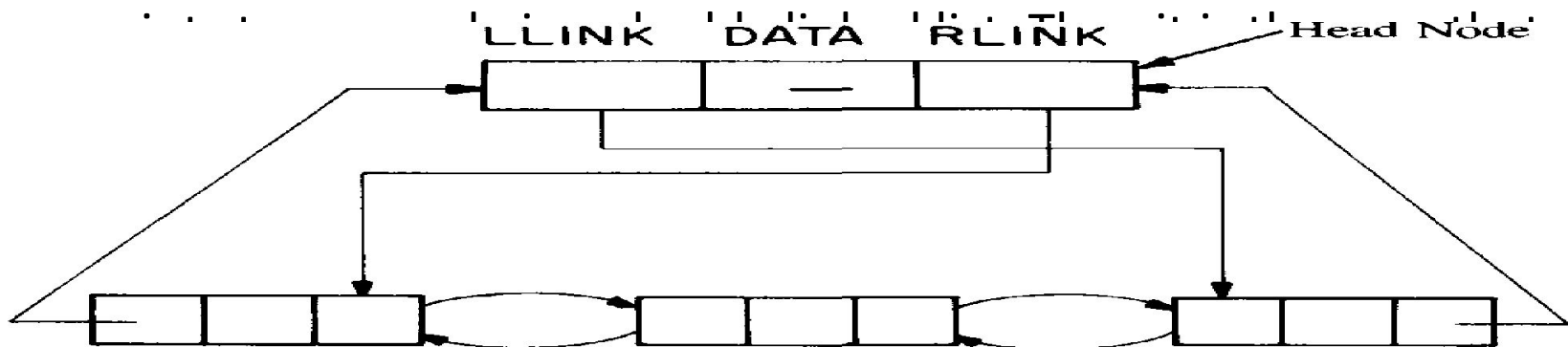


Figure 4.13 Doubly Linked Circular List with Head Node

- This formula reflects the essential virtue of this structure, namely, that one can go back and forth with equal ease.

$$P = \text{RLINK}(\text{LLINK}(P)) = \text{LLINK}(\text{RLINK}(P)).$$

- An empty list is not really empty since it will always have its head node and it will look like



- Now to work with these lists we must be able to insert and delete nodes. Algorithm DDLETE deletes node X from list L .

procedure *DDLETE*(X, L)

if $X = L$ **then call** *NO_MORE_NODES*

 // L is a list with at least one node //

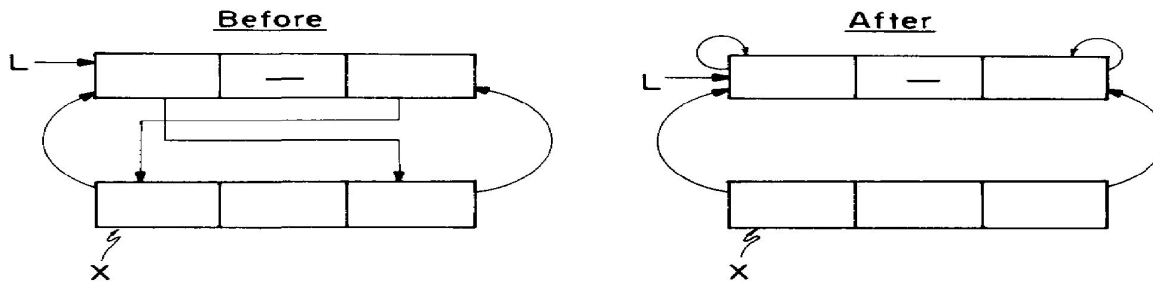
$\text{RLINK}(\text{LLINK}(X)) \leftarrow \text{RLINK}(X)$

$\text{LLINK}(\text{RLINK}(X)) \leftarrow \text{LLINK}(X)$

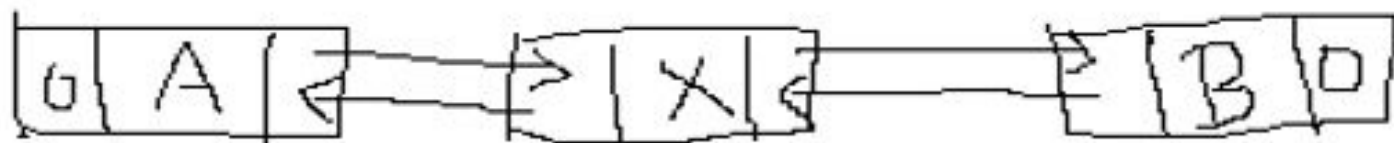
call *RET*(X)

end *DDLETE*

- X now points to a node which is no longer part of list L .
- Let us see how the method works on a doubly linked list with only a single node.



- Even though the RLINK and LLINK fields of node X still point to the head node, this node has effectively been removed as there is no way to access X through L .



L -->

Delete X

```

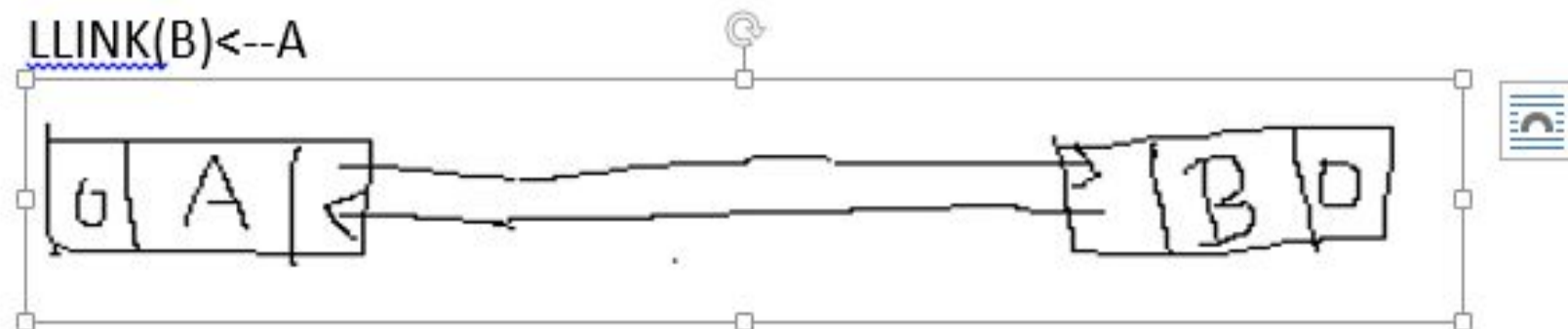
procedure DDLETE(X, L)
  if X = L then call NO_MORE_NODES
  // L is a list with at least one node //
  RLINK(LLINK(X)) ← RLINK(X)
  LLINK(RLINK(X)) ← LLINK(X)
  call RET(X)
end DDLETE
  
```

RLINK(LLINK(X)) ← RLINK(X)

RLINK(A) ← B

LLINK(RLINK(X)) ← LLINK(X)

LLINK(B) ← A



- Insertion is only slightly more complex.

procedure *DINSERT* (*P*, *X*)

 //insert node *P* to the right of node *X*//

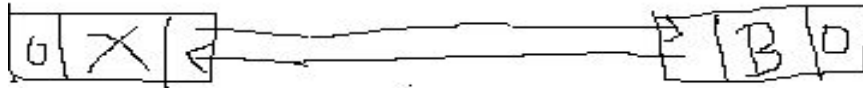
$LLINK(P) \leftarrow X$ //set *LLINK* and *RLINK* fields of node *P*//

$RLINK(P) \leftarrow RLINK(X)$

$LLINK(RLINK(X)) \leftarrow P$

$RLINK(X) \leftarrow P$

end *DINSERT*



procedure *DINSERT* (*P*, *X*)

 //insert node *P* to the right of node *X* //

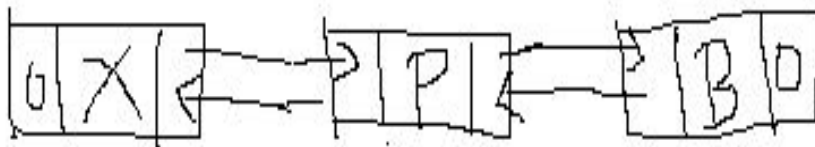
$LLINK(P) \leftarrow X$ //set *LLINK* and *RLINK* fields of node *P* //

$RLINK(P) \leftarrow RLINK(X)$

$LLINK(RLINK(X)) \leftarrow P$

$RLINK(X) \leftarrow P$

end *DINSERT*



4.6 EQUIVALENCE RELATIONS

- Let us put together some of the concepts on linked and sequential representations to solve a problem that arises in the design and manufacture of very large-scale integrated (VLSI) circuits.
- One of the steps in the manufacture of a VLSI circuit involves exposing a silicon wafer using a series of masks.
- Each mask consists of several polygons.
- **Polygons that overlap electrically are equivalent and electrical equivalence specifies a relationship among mask polygons.**
- This relation has several properties that it shares with other equivalence relations, such as the standard mathematical equals.

- Suppose that we denote an arbitrary equivalence relation by the symbol $=$ and that:

(1) For any polygon $x = x$,

that is, x is electrically equivalent to itself. Thus, $=$ is reflexive.

(2) For any two polygons, x and y , if $x = y$ then $y = x$. Thus, the relation $=$ is symmetric.

(3) For any three polygons, x, y , and z , if $x = y$ and $y = z$ then $x = z$. For example, if x and y are electrically equivalent and y and z are also equivalent, then x and z are also electrically equivalent. Thus, the relation $=$ is transitive.

Definition: A relation, $=$, over a set, S , is said to be an *equivalence relation* over S iff it is symmetric, reflexive, and transitive over S .

- Examples of equivalence relations are numerous. For example, the "equal to" ($=$) relationship is an equivalence relation since

(1) $x = x$

(2) $x = y$ implies $y = x$

(3) $x = y$ and $y = z$ implies that $x = z$

- We can use an equivalence relation to **partition a set S** into equivalence classes such that two members x and y of S are in the same equivalence class *iff* $x=y$.
- For example, if we have twelve polygons numbered 0 through 11 and the following pairs overlap:
 $0 = 4, 3 = 1, 6 = 10, 8 = 9, 7 = 4, 6 = 8, 3 = 5, 2 = 11, 11 = 0$
- Then, as a result of the reflexivity, symmetry, and transitivity of the relation $=$, we can partition the twelve polygons into the following equivalence classes:
- $\{0, 2, 4, 7, 11\}; \{1, 3, 5\}; \{6, 8, 9, 10\}$
- These equivalence classes are **important** because they define a signal net that we can use to verify the correctness of the masks.

- The algorithm to determine equivalence works in **two phases**.
- In the **first phase**, we read in and store the equivalence pairs $\langle i, j \rangle$.
- In the **second phase** we begin at 0 and find all pairs of the form $\langle 0, j \rangle$, where 0 and j are in the same equivalence class.
- By transitivity, all pairs of the form $\langle j, k \rangle$ imply that k is in the same equivalence class as 0.
- We continue in this way until we have found, marked, and printed the entire equivalence class containing 0. Then we continue on.

- Our first design attempt appears in Program 4.21.
- Let m and n represent the number of related pairs and the number of objects, respectively.
- We first must figure out which data structure we should use to hold these pairs.
- To determine this, we examine the operations that are required.
- The pair $\langle i, j \rangle$ is essentially two random integers in the range 0 to $n-1$.
- Easy random access would dictate an array, say $\text{pairs}[n][m]$.

- The i th row would contain the elements, j , that are paired directly to i in the input.
- However, this could waste a lot of space since very few of the array elements would be used.
- It also might require considerable time to insert a new pair, $\langle i, k \rangle$, into row i since we would have to scan the row for the next free location or use more storage.

```

void equivalence()
{
    initialize;
    while (there are more pairs) {
        read the next pair  $\langle i, j \rangle$ ;
        process this pair;
    }
    initialize the output;
    do
        output a new equivalence class;
    while (not done);
}

```

Program 4.21: First pass at equivalence algorithm

- These considerations lead us to a **linked list** representation for each **row**.
- Our node structure requires only a data and a link field.
- However, since we still need random access to the i th row, we use a **one-dimensional array**, **$seq[n]$** , to hold the head nodes of the n lists.
- For the second phase of the algorithm, we need a mechanism that tells us whether or not the object, i , has been printed.
- We use the **array $out[n]$** and the constants ***TRUE*** and ***FALSE*** for this purpose.
- Our next refinement appears in Program 4.22.

```
#include <stdio.h>
#include <alloc.h>
#define MAX_SIZE 24
#define IS_FULL(ptr) (! (ptr))
#define FALSE 0
```

```
void equivalence()
{
    initialize seq to NULL and out to TRUE;
    while (there are more pairs) {
        read the next pair, <i,j>;
        put j on the seq[i] list;
        put i on the seq[j] list;
    }
    for (i = 0; i < n; i++)
        if (out[i]) {
            out[i] = FALSE;
            output this equivalence class;
        }
}
```

Program 4.22: A more detailed version of the equivalence algorithm

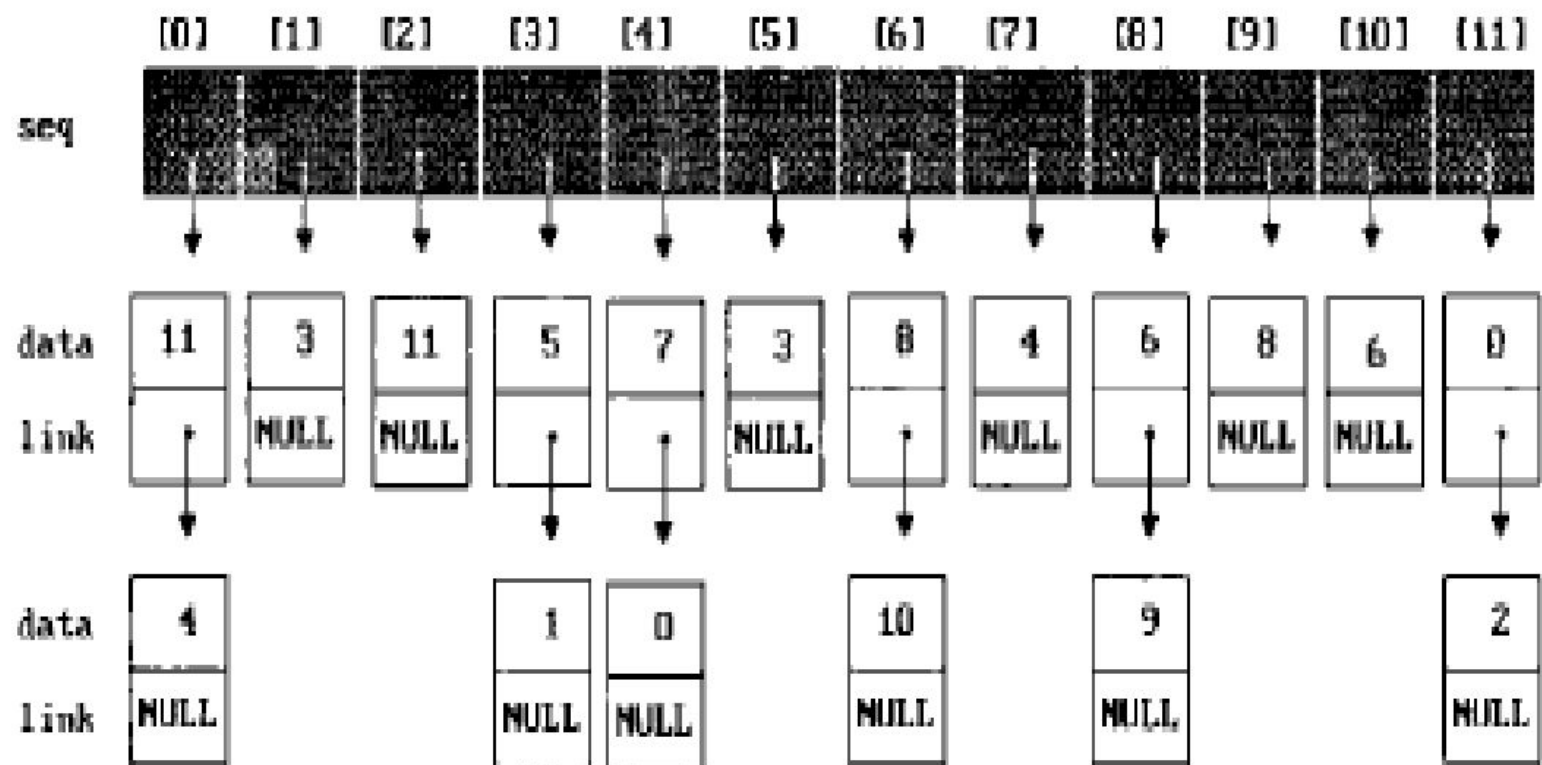


Figure 4.18: Lists after pairs are input

- Let us simulate this algorithm, as we have developed it thus far, using the previous data set.
- After the **while** loop is completed the lists resemble those appearing in Figure 4.18.
- For each relation $i=j$, we use two nodes.
- The variable *seq*[*i*] points to the list of nodes that contains every number that is directly equivalent to *i* by an input relation.
- In phase two, we scan the *seq* array for the first i , $0 \leq i < n$, such that *out*[*i*] = *TRUE*. Each element in the list *seq*[*i*] is printed.
- To process the remaining lists which, by transitivity, belong in the same class as *i*, we create a stack of their nodes.
- We do this by changing the link fields so that they point in the reverse direction. Program 4.23 contains the complete equivalence algorithm.

```

typedef struct node *node_pointer;
typedef struct node {
    int data;
    node_pointer link;
};
void main(void)
{
    short int out[MAX_SIZE];
    node_pointer seq[MAX_SIZE];

    node_pointer x,y,top;
    int i,j,n;

    printf("Enter the size (<= %d) ",MAX_SIZE);
    scanf("%d",&n);
    for (i = 0; i < n; i++) {
        /* initialize seq and out */
        out[i] = TRUE;    seq[i] = NULL;
    }

    /* Phase 1: Input the equivalence pairs: */
    printf("Enter a pair of numbers (-1 -1 to quit): ");
    scanf("%d%d",&i,&j);
    while (i >= 0) {
        x = (node_pointer)malloc(sizeof(node));
        if (IS_FULL(x)) {
            fprintf(stderr,"The memory is full\n");
            exit(1);
        }
        x->data = j;  x->link = seq[i];  seq[i] = x;
        x = (node_pointer)malloc(sizeof(node));
        if (IS_FULL(x)) {
            fprintf(stderr, "The memory is full\n");
            exit(1);
        }
        x->data = i;  x->link = seq[j];  seq[j] = x;
        printf("Enter a pair of numbers (-1 -1 to quit): ");
        scanf("%d%d",&i,&j);
    }
}

```

```

/* Phase 2: output the equivalence classes */
for (i = 0; i < n; i++)
    if (out[i]) {
        printf("\nNew class: %5d",i);
        out[i] = FALSE;    /* set class to false */
        x = seq[i];  top = NULL; /* initialize stack */
        for (;;) {      /* find rest of class */
            while (x) { /* process list */
                j = x->data;
                if (out[j]) {
                    printf("%5d",j);  out[j] = FALSE;
                    y = x->link; x->link = top; top = x; x = y;
                }
                else x = x->link;
            }
            if (!top) break;
            x = seq[top->data]; top = top->link; /*unstack*/
        }
    }
}

```

Program 4.23: Program to find equivalence classes

Applications

Application of Circular Linked List

- There are n people standing in a circle waiting to be executed.
- The counting out begins at some point in the circle and proceeds around the circle in a fixed direction.
- In each step, a certain number of people are skipped and the next person is executed.
- The elimination proceeds around the circle, until only the last person remains, who is given freedom.